

End To End Live Testing Workflow

- [NetworkPeer End-to-End Live Testing Workflow](#)
 - [Prerequisites — Values You Must Provide](#)
 - [Phase 0 — One-Time Bootstrap \(Run Once Per Account\)](#)
 - [Phase 1 — Configure Terraform Inputs](#)
 - [Phase 2 — First Terraform Apply \(Bootstrap Infrastructure\)](#)
 - [Phase 3 — Populate Runtime & Migration Secrets](#)
 - [Phase 4 — First Migration & Service Activation \(ECS Release\)](#)
 - [Phase 5 — Configure Mobile Build Configurations](#)
 - [Android \(`apps/android/networkpeer.production.local.properties` \)](#)
 - [iOS \(`apps/ios/Config.Production.local.xcconfig` \)](#)
 - [Phase 6 — End-to-End Feature Validation Checklist](#)
 - [6.1 Authentication & Session](#)
 - [6.2 Client Job Lifecycle](#)
 - [6.3 Worker Wallet & Evidence](#)
 - [6.4 Notifications & Sync](#)
 - [6.5 Web Dashboard](#)
 - [6.6 Infrastructure Health](#)
 - [Phase 7 — Stripe Webhook Verification](#)
 - [Phase 8 — Rollback & Disaster Recovery Drills](#)
 - [Phase 8 — Mobile Store Release \(When Ready\)](#)
 - [Android \(Google Play\)](#)
 - [iOS \(App Store\)](#)
 - [Appendix A — Local Development with Docker Compose](#)
 - [Appendix B — Common Issues & Fixes](#)
 - [Appendix C — Directory Map for Key Files](#)
 - [Final Sign-Off](#)

NetworkPeer End-to-End Live Testing Workflow

Version: 1.0

Status: Ready for Protected-Environment Execution

Generated: 23 August 2026

Prerequisites — Values You Must Provide

Do not paste secrets into chat. Supply these through your team's approved secret channel (1Password, Bitwarden, HashiCorp Vault, AWS Secrets Manager CLI, etc.).

Category	Required Values	Format / Notes
AWS Account	AWS_ACCESS_KEY_ID , AWS_SECRET_ACCESS_KEY , AWS_REGION (e.g., us-east-1)	Short-lived IAM Identity Center session preferred; static keys only for initial bootstrap.
	Target AWS Account ID	12-digit number
	S3 Evidence Bucket Name	Globally unique, e.g., networkpeer-evidence-prod-addy9087
	Terraform State Bucket Name	Pre-existing, encrypted, versioned
	Terraform Lock Table Name	Pre-existing DynamoDB table
Domain & TLS	API Domain Name	e.g., api.networkpeer.com
	Route 53 Hosted Zone ID	If using Route 53; otherwise provide ACM Certificate ARN
	Web Origin(s) for CORS	e.g., https://app.networkpeer.com
Database	RDS Master Password	AWS manages this; you only need the generated secret ARN after first apply
	Redis AUTH Token	32+ char random string; pass as TF_VAR_redis_auth_token only
Payments (Stripe)	Stripe Publishable Key (Test)	pk_test_... — safe for mobile config
	Stripe Secret Key (Test)	sk_test_... — secret , Secrets Manager only
	Stripe Webhook Secret (Test)	whsec_... — secret , Secrets Manager only
	Stripe Connect Client ID	

Category	Required Values	Format / Notes
		<code>ca_...</code> — Secrets Manager
Push (Firebase)	Firebase Project ID	e.g., <code>networkpeer-prod</code>
	Firebase Client Email	Service account email
	Firebase Private Key	PEM format with <code>\n</code> escaped — secret
	Android <code>google-services.json</code>	Place at <code>apps/android/app/src/production/google-services.json</code>
	iOS <code>GoogleService-Info.plist</code>	Place at <code>apps/ios/NetworkPeer/GoogleService-Info.plist</code> (ignored by git)
SMS/OTP (Twilio)	Twilio Account SID	<code>AC...</code>
	Twilio Auth Token	secret
	Twilio From Number	E.164 format, e.g., <code>+15551234567</code>
Apple / Google	Apple Developer Team ID	10-char ID, e.g., <code>ABCD1234EF</code>
	iOS Bundle ID	e.g., <code>com.networkpeer.mobile</code>
	Android Package Name	e.g., <code>com.networkpeer.mobile</code>
	Play Console Service Account	For CI signing (optional, for automated releases)

Phase 0 — One-Time Bootstrap (Run Once Per Account)

```
# 1. Configure AWS CLI with bootstrap credentials (admin or power user)
```

```
aws configure --profile networkpeer-bootstrap
```

```
# Enter Access Key, Secret Key, Region (e.g., us-east-1), Output: json
```

```
# 2. Create Terraform state bucket (run once)
```

```
aws s3api create-bucket \
```

```
--bucket networkpeer-terraform-state-addy9087 \
```

```
--region us-east-1 \
```

```
--create-bucket-configuration LocationConstraint=us-east-1
```

```
aws s3api put-bucket-versioning \
```

```
--bucket networkpeer-terraform-state-addy9087 \
```

```
--versioning-configuration Status=Enabled
```

```
aws s3api put-bucket-encryption \
```

```
--bucket networkpeer-terraform-state-addy9087 \
```

```
--server-side-encryption-configuration '{"Rules":  
  [{"ApplyServerSideEncryptionByDefault":  
    {"SSEAlgorithm": "AES256"}}]}'
```

```
# 3. Create DynamoDB lock table
```

```
aws dynamodb create-table \
```

```
--table-name networkpeer-terraform-lock \
```

```
--attribute-definitions AttributeName=LockID,AttributeType=S \
```

```
--key-schema AttributeName=LockID,KeyType=HASH \
```

```
--billing-mode PAY_PER_REQUEST \
```

```
--region us-east-1
```

```
# 4. Create evidence bucket (name must be globally unique)
```

```
aws s3api create-bucket \
```

```
--bucket networkpeer-evidence-prod-addy9087 \
```

```
--region us-east-1
```

```
aws s3api put-bucket-versioning \
```

```
--bucket networkpeer-evidence-prod-addy9087 \
```

```
--versioning-configuration Status=Enabled
```

```
aws s3api put-bucket-encryption \
```

```
--bucket networkpeer-evidence-prod-addy9087 \
```

```
--server-side-encryption-configuration '{"Rules":
  [{"ApplyServerSideEncryptionByDefault":
    {"SSEAlgorithm": "AES256"}}]}'

aws s3api put-public-access-block \
--bucket networkpeer-evidence-prod-addy9087 \
--public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublic
```

Phase 1 — Configure Terraform Inputs

```
cd infra/terraform
# Copy examples (these files are gitignored)
cp backend.hcl.example backend.hcl
cp terraform.tfvars.example terraform.tfvars
```

Edit backend.hcl :

```
bucket      = "networkpeer-terraform-state-addy9087"
key         = "networkpeer/prod/terraform.tfstate"
region      = "us-east-1"
dynamodb_table = "networkpeer-terraform-lock"
encrypt     = true
```

Edit terraform.tfvars (only non-secret values):

```

aws_region          = "us-east-1"
environment         = "prod"
project             = "networkpeer"
evidence_bucket_name = "networkpeer-evidence-prod-addy9087"
web_cors_origins    = ["https://app.networkpeer.com"]
|
terraform_state_bucket_name = "networkpeer-terraform-state-addy9087"
terraform_lock_table_name   = "networkpeer-terraform-lock"
|
github_repository = "rudraaxl/NetworkPeer"
github_branch     = "main"
github_environment = "production"
|
availability_zones = ["us-east-1a", "us-east-1b"]
vpc_cidr           = "10.42.0.0/16"
public_subnet_cidrs = ["10.42.0.0/20", "10.42.16.0/20"]
private_app_subnet_cidrs = ["10.42.32.0/20", "10.42.48.0/20"]
private_data_subnet_cidrs = ["10.42.64.0/20", "10.42.80.0/20"]
|
nat_gateway_mode      = "per_az"
enable_interface_vpc_endpoints = true
enable_s3_gateway_endpoint   = true
enable_vpc_flow_logs        = true
|
domain_name          = "api.networkpeer.com"
route53_zone_id      = "Z123456789EXAMPLE" # Your hosted zone ID
acm_certificate_arn   = null
enable_https_listener = true
enable_http_redirect  = true
enable_waf            = true
|
rds_instance_class      = "db.t4g.medium"
rds_allocated_storage_gb = 50
rds_max_allocated_storage_gb = 200
rds_multi_az            = true
rds_backup_retention_days = 30
rds_deletion_protection = true
rds_skip_final_snapshot = false
rds_final_snapshot_identifier = "networkpeer-prod-postgres-final"

```



```
redis_monitoring_interval_seconds = 60
redis_performance_insights_enabled = true
|
redis_node_type                    = "cache.t4g.small"
redis_num_cache_clusters          = 2
redis_automatic_failover_enabled = true
redis_multi_az_enabled            = true
|
alarm_sns_topic_arn = null
|
backup_schedule                = "cron(0 5 ? * * *)"
backup_delete_after_days       = 90
backup_vault_lock_min_retention_days = 30
secret_recovery_window_days     = 30
```

Never put secrets in `terraform.tfvars`. The following are supplied at apply time via environment variables or GitHub Environment secrets: -

`TF_VAR_redis_auth_token` - Runtime/migration secret JSON values
(populated in Secrets Manager after first apply)

Phase 2 — First Terraform Apply (Bootstrap Infrastructure)

```
cd infra/terraform
# Export Redis auth token for this session only (do not save to file)
export TF_VAR_redis_auth_token="$(openssl rand -hex 32)"
# Initialize with remote backend
terraform init -backend-config=backend.hcl
# Format and validate
terraform fmt -check -recursive
terraform validate
# Plan with services parked (no API/worker tasks yet)
terraform plan \
  -var='allow_service_activation=false' \
  -var='api_desired_count=0' \
  -var='worker_desired_count=0' \
  -var='api_min_capacity=0' \
  -var='worker_min_capacity=0' \
  -out=networkpeer-bootstrap.tfplan
# Review the plan carefully. Then apply:
terraform apply networkpeer-bootstrap.tfplan
```

After apply, record these **non-sensitive outputs** in the GitHub Environment production :

Variable	Source
<code>AWS_REGION</code>	<code>var.aws_region</code>
<code>TERRAFORM_ENVIRONMENT</code>	<code>var.environment</code>
<code>TF_STATE_BUCKET</code>	<code>terraform output -raw</code> <code>terraform_state_bucket_name</code>
<code>TF_STATE_KEY</code>	<code>terraform output -raw</code> <code>terraform_state_key</code>
<code>TF_LOCK_TABLE</code>	<code>terraform output -raw</code> <code>terraform_lock_table_name</code>
<code>EVIDENCE_BUCKET_NAME</code>	<code>terraform output -raw</code> <code>evidence_bucket_name</code>
<code>WEB_CORS_ORIGINS_JSON</code>	<code>terraform output -json web_cors_origins</code>
<code>AWS_TERRAFORM_PLAN_ROLE_ARN</code>	<code>terraform output -raw</code> <code>github_actions_plan_role_arn</code>
<code>AWS_TERRAFORM_APPLY_ROLE_ARN</code>	<code>terraform output -raw</code> <code>github_actions_apply_role_arn</code>
<code>AWS_ECS_PUBLISH_ROLE_ARN</code>	<code>terraform output -raw</code> <code>github_actions_deploy_role_arn</code>
<code>ECR_API_REPOSITORY_URL</code>	<code>terraform output -raw</code> <code>api_repository_url</code>
<code>ECR_WORKER_REPOSITORY_URL</code>	<code>terraform output -raw</code> <code>worker_repository_url</code>
<code>ECR_MIGRATOR_REPOSITORY_URL</code>	<code>terraform output -raw</code> <code>migrator_repository_url</code>

GitHub Environment Secret (not variable): - `REDIS_AUTH_TOKEN` = value of `TF_VAR_redis_auth_token` used above

Phase 3 — Populate Runtime & Migration Secrets

After the first apply, two empty Secrets Manager secrets exist. Populate them **out of band** (AWS Console, CLI, or your secret workflow):

Secret: networkpeer/prod/runtime (JSON)

```
{
  "DATABASE_URL": "postgresql://
    networkpeer_app:<APP_PASS>@<RDS_ENDPOINT>:5432/networkpeer?
    sslmode=require",
  "DATABASE_ADMIN_URL": "postgresql://
    networkpeer_admin_api:<ADMIN_PASS>@<RDS_ENDPOINT>:5432/networkpeer?
    sslmode=require",
  "DATABASE_MEDIA_VERIFIER_URL": "postgresql://
    networkpeer_media_verifier:<MEDIA_PASS>@<RDS_ENDPOINT>:5432/
    networkpeer?sslmode=require",
  "DATABASE_FINANCIAL_URL": "postgresql://
    networkpeer_financial_api:<FIN_PASS>@<RDS_ENDPOINT>:5432/
    networkpeer?sslmode=require",
  "REDIS_URL":
    "rediss://:<REDIS_AUTH_TOKEN>@<REDIS_PRIMARY_ENDPOINT>:6379/0",
  "JWT_SECRET": "<64-char-random-hex>",
  "JWT_REFRESH_SECRET": "<64-char-random-hex-different-from-above>",
  "TWILIO_ACCOUNT_SID": "AC...",
  "TWILIO_AUTH_TOKEN": "...",
  "TWILIO_FROM_NUMBER": "+15551234567",
  "AWS_REGION": "us-east-1",
  "AWS_S3_BUCKET": "networkpeer-evidence-prod-addy9087",
  "STRIPE_SECRET_KEY": "sk_test...",
  "STRIPE_WEBHOOK_SECRET": "whsec...",
  "STRIPE_CONNECT_CLIENT_ID": "ca...",
  "PAYMENT_WEBHOOK_SECRET": "<32-char-random>",
  "FIREBASE_PROJECT_ID": "networkpeer-prod",
  "FIREBASE_CLIENT_EMAIL": "firebase-adminsdk-...@networkpeer-
    prod.iam.gserviceaccount.com",
  "FIREBASE_PRIVATE_KEY": "-----BEGIN PRIVATE KEY-----\n...\n-----END
    PRIVATE KEY-----\n",
  "CORS_ORIGINS": "https://app.networkpeer.com",
  "SENTRY_DSN": "https://...@sentry.io/..."
}
```

Secret: `networkpeer/prod/migration` (JSON)

```
{
  "DATABASE_MIGRATION_URL": "postgresql://
    networkpeer_migration:<MIGRATION_PASS>@<RDS_ENDPOINT>:5432/
    networkpeer?sslmode=require",
  "NETWORKPEER_APP_DB_PASSWORD": "<APP_PASS>",
  "NETWORKPEER_ADMIN_DB_PASSWORD": "<ADMIN_PASS>",
  "NETWORKPEER_MEDIA_DB_PASSWORD": "<MEDIA_PASS>",
  "NETWORKPEER_FINANCIAL_DB_PASSWORD": "<FIN_PASS>"
}
```

The RDS master password is managed by AWS in a separate secret (`aws_rds_cluster.master_user_secret`). Do not use it for application workloads.

Phase 4 — First Migration & Service Activation (ECS Release)

The `ECS Release` workflow handles this end-to-end. Trigger it manually from GitHub Actions:

1. Go to **Actions** → **ECS Release** → **Run workflow**
2. Inputs:
 - `target_environment` : `production`
 - `confirmation` : `release`
3. The workflow will:
 - Build & push 3 immutable images (API, Worker, Migrator) tagged with Git SHA
 - Apply Terraform with `allow_service_activation=false` (services parked at 0)
 - Run the one-shot Migrator task (runs migrations + provisions least-privilege DB roles)
 - Verify migrator exit code = 0
 - Apply Terraform with `allow_service_activation=true` + SHA tags
 - Verify ECS services stable, ALB targets healthy
 - Call `https://api.networkpeer.com/api/v1/live` and `/api/v1/health` — both must return HTTP 200

If any step fails, the workflow restores the pre-release task definitions and capacity. An initial bootstrap (no prior release) is restored to zero capacity.

Phase 5 — Configure Mobile Build Configurations

Android (`apps/android/networkpeer.production.local.properties`)

```
NETWORKPEER_PRODUCTION_API_BASE_URL=https://api.networkpeer.com/api/v1/
NETWORKPEER_PRODUCTION_STRIPE_PUBLISHABLE_KEY=pk test ... # or pk live for prod
NETWORKPEER_PRODUCTION_REALTIME_URL=https://api.networkpeer.com
NETWORKPEER_PRODUCTION_REALTIME_ORIGIN=https://app.networkpeer.com
```

Place `google-services.json` at `app/src/production/google-services.json`.

iOS (`apps/ios/Config.Production.local.xcconfig`)

```
NETWORKPEER_API_BASE_URL = https://api.networkpeer.com/api/v1/
NETWORKPEER_STRIPE_PUBLISHABLE_KEY = pk test ... # or pk live
NETWORKPEER_SOCKET_IO_ENABLED = YES
NETWORKPEER_MAX_EVIDENCE_BYTES = 26214400
```

Place `GoogleService-Info.plist` at `NetworkPeer/GoogleService-Info.plist`.

Phase 6 — End-to-End Feature Validation Checklist

Run these **manually on staging/production** after the ECS Release succeeds.
Mark each ☒ when verified.

6.1 Authentication & Session

Test	Expected
Client OTP request (dev mode)	Code echoed in response
Client OTP request (prod mode)	SMS delivered via Twilio / Email via SES
Client OTP verify → JWT pair	Access + Refresh tokens stored in Keystore/Keychain
Access token expiry (15m) → auto-refresh	Single refresh flight; no duplicate 401 loops
Refresh token rotation	Old token invalidated; new token stored
Client logout	Refresh family revoked; device token deregistered; local session cleared
Worker logout	Same as client
Inactive/suspended user	403 on all authenticated endpoints

6.2 Client Job Lifecycle

Test	Expected
Create job (GeoJSON [lon, lat] , checklist, idempotency key)	Job FUNDING , UNFUNDED escrow
Fund job → Stripe PaymentIntent	PaymentSheet opens; client_secret returned
Stripe test payment (card 4242 4242 4242 4242)	Webhook → escrow HELD , job POSTED
Worker discovers nearby (privacy-safe)	No client identity, no exact address, coarse distance band
Worker accepts job (atomic)	Job ASSIGNED , worker VERIFIED required
Worker advances EN_ROUTE → AT_LOCATION → IN_PROGRESS	One legal transition at a time
Capture evidence (camera/gallery/file)	Local SHA-256, reserve → presigned POST → confirm
Submit work (all required evidence UPLOADED)	Job SUBMITTED
Client reviews evidence (download via short-lived URL)	HTTPS URL, Cache-Control: no-store , no bucket/key exposed
Client approves	Escrow RELEASED , ledger postings, payout dispatched
Client completes	Job COMPLETED , payout COMPLETED after provider confirmation

6.3 Worker Wallet & Evidence

Test	Expected
Worker wallet shows <code>available_cents</code> , <code>pending_cents</code> , <code>lifetime_cents</code>	Balances match ledger
Evidence retry (network failure)	Retries with same idempotency key; no duplicate S3 objects
Large file (>25 MiB) rejected	Client-side guard + server reservation rejection
Invalid MIME type rejected	Client-side + server-side validation
WebM audio vs video correctly typed	<code>audio/webm</code> vs <code>video/webm</code> in reservation

6.4 Notifications & Sync

Test	Expected
FCM data-only push received (Android)	<code>recipient_user_id</code> matches active session → local notification + sync
APNs background push received (iOS)	<code>content-available: 1</code> → local notification + foreground sync
Push for logged-out account	Discarded locally; no notification shown
Socket.IO <code>sync:ready</code> / <code>sync:event</code>	Triggers HTTP <code>/sync</code> or <code>/worker/sync</code> reconciliation
Cursor persistence across app restart	Sync resumes from last cursor; no duplicate events
Notification read / read-all	Local + remote state reconciled

6.5 Web Dashboard

Test	Expected
Client dashboard: create/fund/approve/complete/dispute	All actions work; real-time updates via Socket.IO
Worker dashboard: nearby/accept/advance/submit	All actions work; real-time updates
Admin dashboard (web only): worker verification, analytics, audit log	Accessible only to <code>ADMIN</code> role

6.6 Infrastructure Health

Test	Expected
<code>GET /api/v1/live</code>	HTTP 200 <code>{status:"live"}</code>
<code>GET /api/v1/health</code>	HTTP 200 <code>{status:"ok", database:true, postgis:true}</code>
ALB target group	All targets <code>healthy</code>
ECS services	<code>RUNNING</code> count = <code>DESIRED</code> count; <code>PENDING</code> = 0
CloudWatch alarms	No <code>ALARM</code> state; all <code>OK</code> or <code>INSUFFICIENT_DATA</code>
RDS automated backup	Latest snapshot < 24h old
AWS Backup recovery point	Latest recovery point < 24h old
S3 evidence bucket	Versioning enabled, default encryption AES-256, block public access

Phase 7 — Stripe Webhook Verification

```
# 1. Get the Stripe webhook endpoint URL from Terraform output
WEBHOOK_URL="https://api.networkpeer.com/api/v1/payments/webhook"
[REDACTED]

# 2. In Stripe Dashboard → Developers → Webhooks, add endpoint:
#   URL: <WEBHOOK_URL>
#   Events: payment_intent.succeeded, payment_intent.payment_failed,
#           payment_intent.canceled, charge.refunded, payout.paid,
#           payout.failed
[REDACTED]

# 3. Test with Stripe CLI (local tunnel)
stripe listen --forward-to localhost:3000/api/v1/payments/webhook
# Use the provided webhook secret for local .env
```

Phase 8 — Rollback & Disaster Recovery Drills

Drill	Procedure	Success Criteria
API Rollback	<code>gh workflow run ecs-release.yml -f confirmation=release -f target_environment=production</code> with previous SHA	Previous task definition active; API healthy
Worker Rollback	Same as above	Worker tasks on previous revision
RDS Point-in-Time Restore	AWS Console → RDS → Restore to point in time (5 min ago)	New instance accessible; data consistent
AWS Backup Restore	AWS Backup → Restore recovery point → New DB instance	Data matches point-in-time
Secret Rotation	Generate new JWT secrets → Update Secrets Manager → Trigger ECS Release	New tokens issued; old tokens rejected
Redis Failover	Reboot primary node (if Multi-AZ)	Replica promotes; API reconnects automatically
S3 Evidence Recovery	Delete current version → Restore from version history	Evidence download works via API

Phase 8 — Mobile Store Release (When Ready)

Android (Google Play)

1. Generate signed AAB: `./gradlew bundleProductionRelease` (requires signing config in CI)
2. Upload to Play Console → Internal Testing
3. Test on physical devices (various API levels, screen sizes)
4. Promote to Closed → Open → Production

iOS (App Store)

1. Open `NetworkPeer.xcodeproj` in Xcode
2. Select **Any iOS Device** → **Product** → **Archive**
3. Validate → Distribute App → TestFlight
4. Test on physical devices (iPhone SE, 15, 15 Pro, iPad)
5. Submit for Review

Privacy Manifest must declare: Location, Camera, Photos, Notifications, Identifiers.

Appendix A — Local Development with Docker Compose

```
cd NetworkPeer-main  
  
# Create .env.prod with test values (never commit)  
cat > .env.prod <<'EOF'  
POSTGRES_PASSWORD=postgres_password  
NETWORKPEER_APP_DB_PASSWORD=app_password  
NETWORKPEER_ADMIN_DB_PASSWORD=admin_password  
NETWORKPEER_MEDIA_DB_PASSWORD=media_password  
NETWORKPEER_FINANCIAL_DB_PASSWORD=financial_password  
REDIS_PASSWORD=redis_password  
JWT_SECRET=0123456789abcdef0123456789abcdef  
JWT_REFRESH_SECRET=fedcba9876543210fedcba9876543210  
TWILIO_ACCOUNT_SID=AC1234567890  
TWILIO_AUTH_TOKEN=twilio_token  
TWILIO_FROM_NUMBER=+15550000000  
AWS_REGION=us-east-1  
AWS_ACCESS_KEY_ID=AKIATESTING  
AWS_SECRET_ACCESS_KEY=testing_secret  
AWS_S3_BUCKET=networkpeer-compose-test  
PAYMENT_WEBHOOK_SECRET=0123456789abcdef0123456789abcdef  
STRIPE_SECRET_KEY=sk_test_example  
STRIPE_WEBHOOK_SECRET=whsec_example  
STRIPE_CONNECT_CLIENT_ID=ca_example  
CORS_ORIGINS=http://localhost:3001,http://localhost:5173  
EOF  
  
# Start stack (migrate → api → worker)  
docker compose -f docker-compose.prod.yml up --build -d  
  
# View logs  
docker compose -f docker-compose.prod.yml logs -f api  
docker compose -f docker-compose.prod.yml logs -f worker
```

Appendix B — Common Issues & Fixes

Symptom	Cause	Fix
<code>terraform plan</code> fails: <code>error validating provider credentials</code>	AWS credentials not configured or expired	Re-run <code>aws configure</code> or refresh session
<code>ECS Release</code> fails at migration: <code>exitCode=1</code>	Migration SQL error or DB connection	Check CloudWatch log group <code>/ecs/networkpeer-prod/migration</code>
<code>ECS Release</code> fails at health check	ALB targets never healthy	Check security groups (ALB→API port), API logs, <code>trustProxy</code> config
Android build fails: <code>SDK location not found</code>	<code>ANDROID_HOME</code> not set	<code>export ANDROID_HOME=/opt/homebrew/share/android-commandlinetools</code>
iOS <code>xcodebuild</code> fails: <code>requires Xcode</code>	Only Command Line Tools installed	Install full Xcode from App Store
Push notifications not received	Firebase config missing or wrong bundle ID	Verify <code>google-services.json</code> / <code>GoogleService-Info.plist</code> match bundle ID
Stripe webhook 400	Wrong secret or endpoint	Verify <code>STRIPE_WEBHOOK_SECRET</code> matches Stripe Dashboard
<code>redis_auth_token</code> mismatch	Token in Terraform ≠ token in Secrets Manager	Use same value for <code>TF_VAR_redis_auth_token</code> and <code>REDIS_URL</code>

Appendix C — Directory Map for Key Files

Need	File
Backend routes & contracts	<code>NetworkPeer-main/src/routes/</code> , <code>src/contracts.ts</code>
Database migrations	<code>NetworkPeer-main/migrations/*.sql</code>
DB roles provisioning	<code>NetworkPeer-main/scripts/provision-app-role.sql</code>
S3 evidence handling	<code>NetworkPeer-main/src/services/media-storage-service.ts</code>
Client evidence review	<code>NetworkPeer-main/src/services/client-evidence-review-service.ts</code>
Push notification gateway	<code>NetworkPeer-main/src/services/push-notification-service.ts</code>
Android app	<code>apps/android/app/src/main/java/com/networkpeer/mobile/</code>
iOS app	<code>apps/ios/NetworkPeer/</code> , <code>apps/ios/Sources/NetworkPeerCore/</code>
Mobile contract	<code>packages/api-contracts/networkpeer-mobile-contract.yaml</code>
Terraform AWS target	<code>infra/terraform/</code>
CI/CD workflows	<code>.github/workflows/*.yaml</code>
Technical DOCX	<code>docs/NETWORKPEER_END_TO_END_DELIVERY_GUIDE.docx</code>
Executive DOCX	<code>docs/EXECUTIVE_AWS_INFRASTRUCTURE_JUSTIFICATION_GUIDE.docx</code>

Final Sign-Off

All automated checks pass. The repository is ready for live deployment once the protected-environment values above are supplied. No code changes are required.

Role	Name	Approval	Date
Engineering Lead			
Platform/Cloud Owner			
Security Owner			
Finance/Budget Owner			
Mobile Release Owner			
QA Owner			